

Research directions for verifiable AI slowdown

A self-contained technical map of the main public papers and posts on verifying (and preferably throttling) frontier AI training under low trust. For each piece: who wrote it, what problem it addresses, how mature it is, and how it relates to the rest of the stack. Diagrams first; public links in each section.

2026-08-04 · standalone briefing · sources linked inline
(arXiv / LessWrong / Substack / project sites)

0. Why this exists, what “Claim A” means, and the stack

0.0 Motivation – why anyone is building this

Anthropic (2026), *When AI builds itself*. The Institute argued the world should have the *option* of a coordinated slowdown or temporary pause on frontier development — but only if parties can **verify** that others actually stopped, so bad actors cannot use a collective pause as cover to race ahead. Conditional pause language is the point: verification systems are a prerequisite, not a side quest. (anthropic.com/institute/recursive-self-improvement)

AI 2027, Appendix S (scenario research). If the US and China tried an AI arms-control style deal, how would cheating be caught? The appendix lists four imperfect families: (1) intelligence collection, (2) crude compute shutdowns with inspectors, (3) **hardware-enabled mechanisms (HEM)** — tamper-resistant reporting of what accelerators run, analogous to seals/attestations in other verification regimes, and (4) politically extreme “AI lie detection” of officials. This deck is mostly fleshing out family (3) and the monitoring stack that sits under it. (ai-2027.com · research notes on compute / verification)

Same problem, different genres: a lab institute essay (Anthropic) and a public scenario appendix (AI 2027) both treat **credible verification** as the binding constraint on any pause story.

0.1 The policy problem in one paragraph

Labs and governments sometimes talk about a **coordinated slowdown or pause** on frontier model development. That option is worthless if parties cannot tell whether others actually stopped. Self-reports are not enough under low trust (rival states, or competing labs). The technical agenda is: build monitoring / attestation / enforcement so large unauthorized *training* is discoverable or physically hard — without requiring mutual trust in each other’s word.

Training vs inference is the load-bearing distinction. **Training** updates weights (often with heavy multi-GPU communication). **Inference** serves a fixed model. Datacenters can look “busy” either way; a sophisticated actor can try to disguise training (especially post-deployment RL) as serving. So “the GPUs are warm” is not a proof of compliance.

0.2 Four claims people mix up

People say “AI verification” for several *different sentences*. This deck is almost entirely about **Claim A**.

Four different sentences people call “AI verification” – only one is this deck’s focus

Claim A No covert frontier training / only allowed workloads Who cares: treaties, pause / slowdown, export controls THIS DECK	Claim B You got the model you were sold / audited Who cares: banks, hospitals, buyers Tech: identity, recompute, DRM-ish	Claim C Run happened in this jurisdiction / TEE; cloud can’t peek Who cares: sovereignty, GDPR, enterprise Tech: CC / TEE, attestation, location	Claim D This agent only did approved tool calls Who cares: agent users / enterprises Tech: audit trails, policy engines
---	--	--	---

Mixing claims is the most common briefing failure. FlexHEG / Cankaya / joshc / Rahman are mostly Claim A. Commercial TEE “AI Passport” products are mostly B/C (sometimes D) — useful, but they do not by themselves prove “no covert training.”

CLAIM	PLAIN ENGLISH	PRIMARY AUDIENCE	EXAMPLE TECH
A	No covert frontier training; only allowed / declared workloads on monitored compute	Treaties, pause/slowdown, export-control regimes	TAPs, recompute, train↔infer detectors, FlexHEG, PoL-style transcripts
B	The model that answered you is the one you bought / audited	Enterprise buyers, regulated sectors	Model identity binding, inference receipts, DRM-adjacent ZK
C	This run happened in a stated place / TEE; the host can’t read your data	Sovereignty, GDPR, confidential cloud	Confidential computing, remote attestation, location proofs
D	This agent only performed approved tool calls	Agent products / enterprise copilots	Audit logs, policy engines, “passport” receipts

SCOPE OF THIS DECK

In: Claim A mechanisms and the research lineage around them. **Out (except as contrast):** commercial TEE inference products aimed at B/C/D. Those can share primitives (attestation, recompute) but answering B/C does not answer “did you secretly train a bigger model?”

0.3 Map of works covered

Claim A lineage — arrows = motivates / hardens / implements a slice (not call-stack)

Numbers = deck reading order. Same row = siblings (different tools), not “later replaces earlier.”



FlexHEG + Cankaya: longer silicon track vs near-term facility retrofit — complementary, not sequential.

Wasil orients. Shavit designs classical monitoring; Rahman / Peigné harden thin slices of it (telemetry vs ZK transcripts). joshc stress-tests cheap facility controls → Cankaya assembles the retrofit; zkLLM/VerInf/DiFR sit under Plan B evaluate. FlexHEG is the parallel hardware-enforcement track.

loose	NTM / inspections — satellites, energy, site visits (Wasil’s access-light methods)
medium	Network / facility controls — TAPs, bandwidth caps, memory wipes, output recompute (joshc; Cankaya taps)
tight	Workload monitoring — weight snapshots + transcripts (Shavit); thin train↔infer telemetry (Rahman)
hard	Hardware-enabled guarantees — FlexHEG FLOP caps / licenses / interlocks
crypto	ZK training proofs — Peigné et al. (§9 research horizon)
infer	Near-term inference retrofit — Cankaya Plan A (§5 taps + bit-exact) then Plan B (§6 zkLLM · §7 VerInf)

Maturity legend used below: *Survey* (conceptual menu) · *Blueprint* (architecture, no full deploy) · *Empirical paper* (measured results) · *Prototype / demo* · *Horizon* (no working end-to-end system yet) · *Product* (mass deployment — almost nothing here qualifies).

PRIMER

0b. Hardware · ML systems · crypto vocabulary

Zoom from hall → interconnect layers → rack → node → board → SM, then why traffic looks different for train vs infer, then ZK vocab for §§6–7. Skim once; jump back when a later section feels opaque.

1 · Where this lives – a real hall



Real AI cluster hall (Meta AI Research SuperCluster press photo): rack rows, cold-aisle containment, overhead cable trays carrying the scale-out fabric. Source: Meta AI — Introducing the AI Research SuperCluster.

2 · Organizing abstraction – interconnect ladder

Everything below is a zoom into one of these layers. Tap feasibility is uneven because only some layers ever become light.

Interconnect ladder – copper vs optics (organizing abstraction for everything below)

Read top→bottom once; later figures are zooms of a single layer.

1 • Die / package / board

On-die mesh • HBM • NVLink-C2C (Grace↔GPU). Electrical, mm-cm. Opaque to facility taps. → zoom: board + \$M

2 • Scale-up domain (NVLink / NVSwitch)

Intra-node mesh or whole-rack copper spine (NVL72). Still mostly electrical. → zoom: NVL72 + 8-GPU node

3 • Scale-out fabric (InfiniBand / RoCE / Spectrum-X)

NIC → DAC copper (short) or AOC/transceiver (longer). Here electrical ↔ optical conversion happens: SerDes → laser/photodiode in QSFP/OSFP → fiber → far end recovers electrical → switch ASIC.

Tap-friendly

Optical fiber / NS chokepoints • passive taps

Tap-hostile

Copper NVLink spine • on-board links

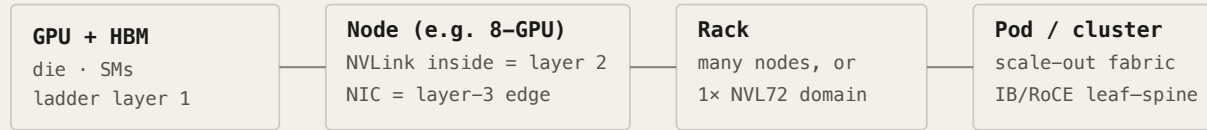
IB/RoCE (layer 3) carry the cross-node all-reduces Claim A people want to meter. Layer-2 copper spines move TB/s without becoming light — hard tap case for Cankaya/joshc.

LINK	MEDIUM	JOB	TAP NOTE
NVLink / NVSwitch	Copper (board / cable cartridge / rack spine)	Scale-up: activations, TP shards, in-rack collectives	Hard — never becomes light
InfiniBand (Quantum...)	DAC copper short; fiber + QSFP/OSFP longer	Scale-out GPU↔GPU (NCCL over RDMA)	Fiber segments = tap candidates
RoCE / Spectrum-X	Same physical story as IB, Ethernet framing	Same role as IB in many clouds	Same EO story
PCIe	Board traces	GPU↔CPU / GPU↔NIC	Inside box
AOC / transceiver	Electrical→laser→fiber→photodiode→electrical	Longer runs, leaf-spine, NS exit	Conversion lives in the pluggable module

Electro-optical conversion in one sentence: the NIC SerDes is electrical; for distance you plug in a module that drives a laser into fiber and recovers bits at the far end. Passive optical taps copy photons on the fiber; they do nothing for copper NVLink spines inside the rack.

3 · Ladder mapped onto a facility

Datacenter hierarchy – IB/RoCE ≠ “only cross-pod”



Where IB/RoCE actually starts = leaving the NVLink domain

Classic DGX: NVLink = one 8-GPU node → even same-rack node↔node is already IB/RoCE.
NVL72: NVLink = whole rack (72 GPUs) → IB/RoCE starts rack↔rack (in-pod or cross-pod).

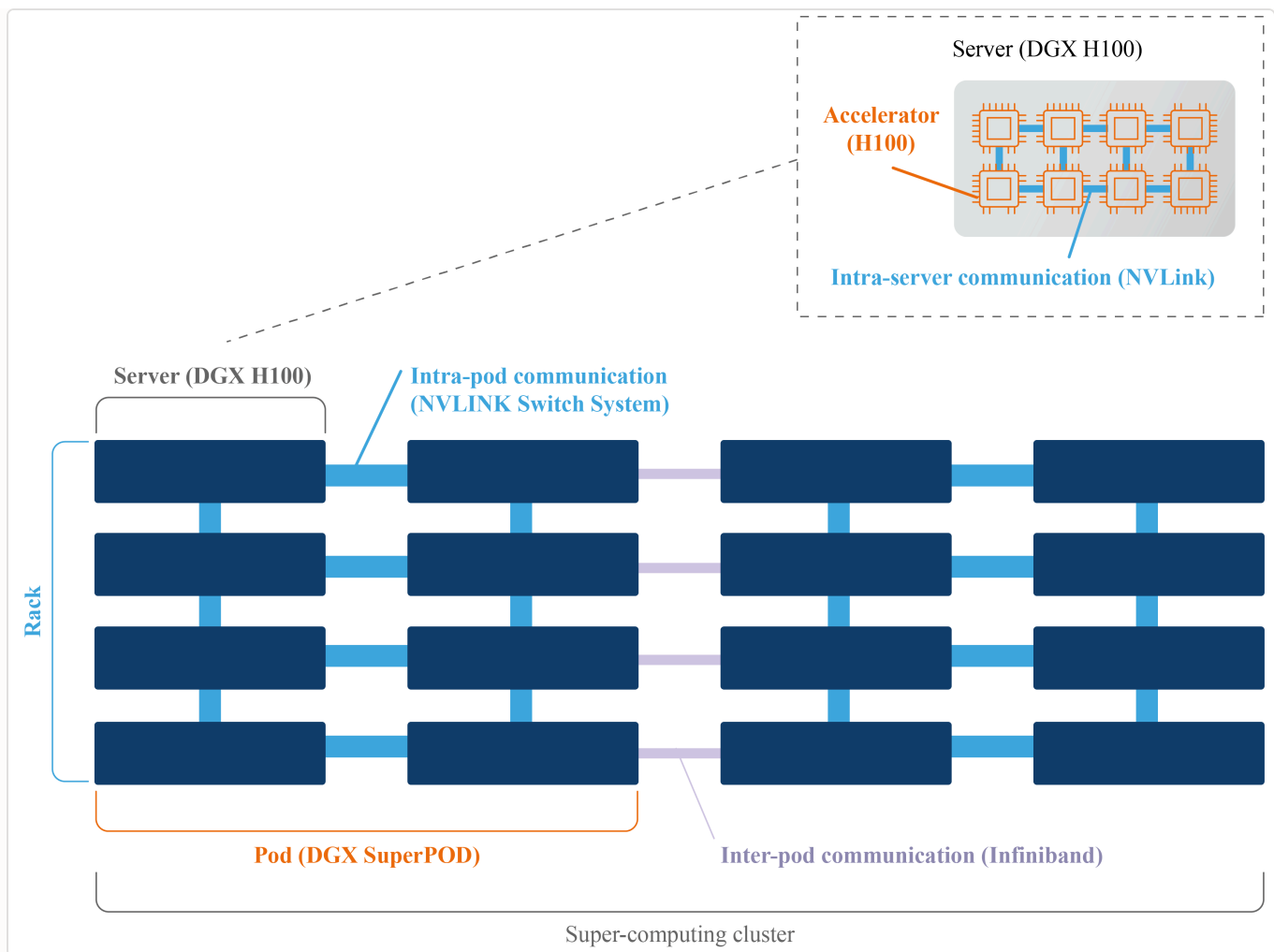
East-West (inside the fence)

GPU↔GPU grads / acts / KV
= NVLink (in-domain) + IB/RoCE (out)
Fat · hard to tap fully

North-South (to the outside)

API tokens, SSH, admin, uploads
Skinny · natural chokepoint
Best first place for taps

NIC sits on the node because that is the scale-out edge for classic topologies — not because IB is “a pod thing.” Cross-pod is just one hop on the same scale-out fabric.



Same hierarchy, drawing form: servers → racks → pods → site. East–West ≈ ladder layers 2–3 inside the fence; North–South exits the site (best first tap chokepoint).

4 · Zoom: one scale-up domain (rack)



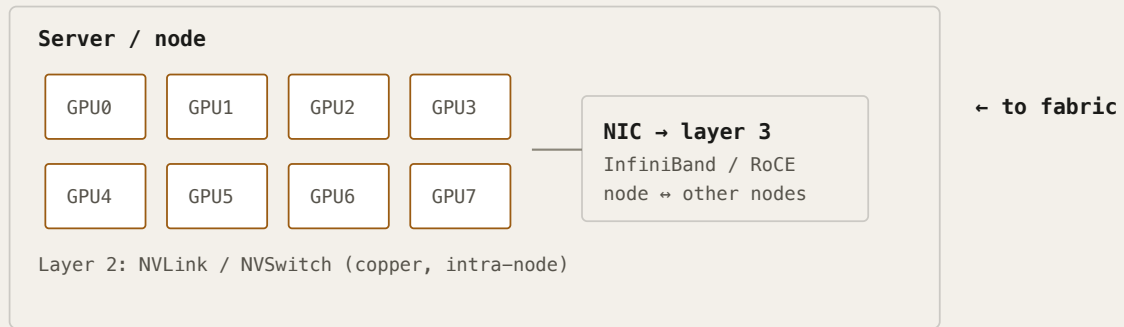
Ladder layer 2 at modern density: GB200 NVL72 = 72 Blackwell GPUs + Grace CPUs in one liquid-cooled copper NVLink domain (~130 TB/s). Source: NVIDIA Blog.



What layer-2 cabling looks like up close: dense copper (not fiber). Optical conversion mostly starts at the scale-out NIC edge (layer 3). Source: NVIDIA Blog.

5 • Zoom: one node (ladder layers 2→3)

One 8-GPU node = ladder layers 2→3 (spatial zoom)

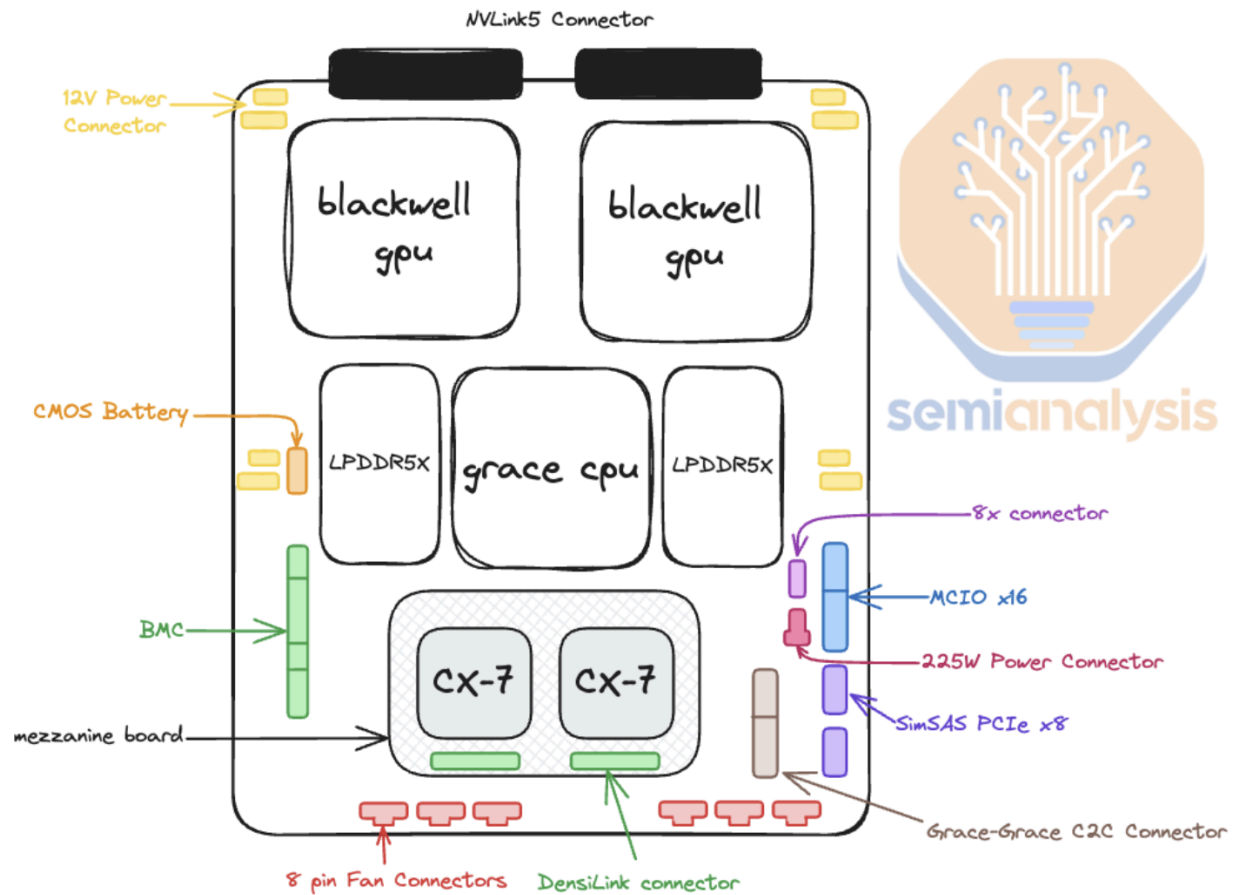


NVML reads per-GPU sensors inside the box. Cross-node bytes = NIC/NCCL path (layer 3), not NVML alone.

Spatial zoom of the interconnect ladder's layers 2→3. Fowler-style "two 8-GPU nodes" watch the cable between NICs; Rahman-style classifiers watch per-GPU NVML time series.

6 · Zoom: one board (ladder layers 1→2)

GB200 Bianca Board



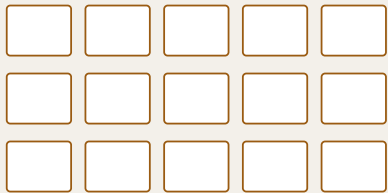
Source: SemiAnalysis

GB200 "Bianca" board (SemiAnalysis). Layer-1/2 callouts: **Blackwell x2** — AI compute dies (SMs + HBM on-package); **Grace CPU** — Arm host / orchestrator; **LPDDR5X** — Grace system DRAM (not GPU HBM); **NVLink-C2C** — Grace↔GPU coherent link (still layer 1); **NVLink5 connectors** — out to rack scale-up domain (layer 2); **CX-7** — NIC mezzanine → IB/Ethernet scale-out (layer 3; EO starts here on long runs); **Grace-Grace C2C** — board-to-board CPU link; **BMC** — out-of-band management; **MCIO / SlimSAS** — PCIe storage/expansion. FlexHEG near-term retrofits often target the *NIC / enclosure*, not a GPU-die respin.

7 · Zoom: inside the GPU die – SM

SM = Streaming Multiprocessor (GPU's compute tile)

One GPU die (cartoon)



H100 ≈ 132 SMs · each SM = many
CUDA cores + Tensor Cores + schedulers

What “SM util” actually measures

NVML “SM utilization” ≈ fraction of SMs that
had ≥1 warp active over the sample window.

High SM util ⇒ compute units busy.

Low SM util + high HBM BW ⇒ memory-bound
(classic decode / bandwidth-limited kernels).

Does **not** say train vs infer, legal vs covert,
or which model. Just “are SMs awake?”

Think of SMs as many small CPUs glued to shared HBM. Prefill matmuls wake lots of
SMs; token-by-token decode often waits on memory → SM util can look “idle” while the
GPU is still serving.

8 · Why so many GPUs, and why traffic differs

Frontier training is big because the *model state does not fit on one GPU*, so you must shard
and communicate constantly — mostly on ladder layers 2–3.

SCALE INTUITION (PUBLIC ANCHORS, ~2024–26)	NUMBER
Llama 3.1 405B dense params	405×10 ⁹
BF16 weights alone	405B × 2 B ≈ 810 GB
Naive train state (W + grad + Adam ≈ 16 B/param)	≈ 6.5 TB total
One H100 HBM	80 GB (SXM) / ~94–96 GB (some SKUs)
B200 / Blackwell HBM class	~180–192 GB/GPU (gen-dependent)
One GB200 NVL72 rack	72 GPUs · ~13.4 TB HBM aggregate
Llama 3.1 405B pretrain cluster (Meta report)	up to ~16,384 H100s · ~3.8×10 ²⁵ FLOP · ~15.6T tokens
Rough “why many GPUs”: fit train state + activations	405B needs ≫1 GPU; production runs use 10 ³ – 10 ⁴ +

Closed frontier labs (OpenAI / Anthropic / Google) don’t publish exact GPU counts; treat
Llama 405B as the best public lower-bound picture of “what a near-frontier dense run looks
like,” not the ceiling. MoE frontier runs can advertise huge total params with fewer active
params/token — still multi-thousand-GPU jobs.

Collectives that make training “loud” on the network

All-reduce (DDP heartbeat)

Each GPU has local gradient g_i
→ everyone gets $\text{mean}(g)$
Bytes / step $\approx O(\text{model size})$
Big, regular, every step
Cross-node → IB/RoCE counters spike

All-gather (FSDP / some TP)

Each GPU holds a shard
→ assemble full tensor locally
Also $O(\text{params})$ class traffic
Inference usually ships activations
or KV — $O(\text{tokens} \times \text{hidden})$, not $O(\text{params})/\text{step}$

DiLoCo / LocalSGD: train locally for many steps, sync rarely → average cross-node GB/s can look “inference-quiet” while training still happens.

Inference anatomy: prefill vs decode (and why disagg ships KV)

Prefill — ingest whole prompt, build KV

Decode — one new token at a time

Same box (usual serving)

Prefill+decode share GPUs
Cross-node often quiet

Disaggregated serving

Prefill pool → ship KV → decode pool
Can make NS/EW look “training-loud”

SM util: how busy GPU compute units are. Prefill can pin SMs; decode is often memory-bandwidth bound — so SM util alone is a weak train/infer label.

TERM	MEANING
TP	Tensor parallelism — split a layer’s matrices across GPUs; exchange activations each layer
DP / DDP	Data parallel — full model replica per worker; all-reduce gradients
FSDP	Fully sharded data parallel — shard params; all-gather when needed
NCCL	NVIDIA collective communications library implementing all-reduce etc.
NVML / DCGM	Per-GPU sensor APIs (power, SM util, memory...). Not cross-node byte counters
SM util	Fraction of SMs busy in the sample window — compute busyness, not “is this training?”
Warden	Active network scrubber on the monitored path (normalize headers/timing to kill stego) — see §4

TERM	MEANING
DiLoCo	Low-communication training: many local steps, rare outer sync — can evade “fat interconnect” detectors

9 · Zero-knowledge proofs (for §§6–7, §9)

Four properties (use these words precisely in §§6–8)

Completeness

Honest compliant prover almost always passes. False reject → kills the deal.
 (“If I’m clean, I get accepted.”)

Soundness

Cheater almost never passes.
 False accept → fake compliance.
 (“Dirty run can’t forge Accept.”)

Confidentiality

Weights / prompts / activations / customer data stay private from the other party (and often from each other’s silicon).
 ZK ≈ cryptographic confidentiality of witness.

Assurance

How sure is the auditor that cheating would be caught in the field?
 Includes soundness + sampling rate + tap coverage + stego slack (joshc ~1%).

Plan A vs Plan B on these axes

Plan A (plaintext AE recompute): can be strong on completeness/soundness/assurance if bit-exact + dense sample but burns confidentiality (weights + challenged I/O enter the AE). Plan B (ZK): keep soundness/completeness, restore confidentiality of the witness; assurance still needs sampling + good taps — π doesn’t watch untapped

Don’t collapse these. A demo can be complete+sound in crypto and still give low assurance if you sample 0.001% of requests or leave EW copper blind.

ZK vocabulary (enough to read zkLLM / VerInf / Peigné)

Statement

"I ran model W on x
and got y" (or a bound)

Public: arch, C=Commit(W)

Witness

Secret intermediates:
weights, activations...

Prover holds; verifier doesn't

Constraints

Algebraic equations that
every correct step must
satisfy (the "circuit")

Properties you want

Completeness: honest prover convinces

Soundness: cheater almost never passes

Zero-knowledge: proof leaks ~nothing extra

In this literature

zkLLM: prove exact integer circuit

VerInf: prove unexplained bits $\leq U$

Peigné: ZK for training steps (horizon)

The integer / finite-field constraint (why float GPUs $\neq$ ZK circuits)

SNARKs do arithmetic in a prime field $\mathbb{F}_p$ (integers mod p). GPU serving does BF16/FP8 floats with rounding. So zkLLM-style systems re-express the forward pass as a quantized / integer circuit, then prove that circuit. What you proved: "integer program with Commit(W) produced y." What you did **not** prove: bit-exact match to arbitrary float serving on H100/B200. Softmax/nondeterminism $\rightarrow$ lookups + tolerances, not native IEEE floats. VerInf dodges this by proving an unexplained-info bound instead of exact float equality.

A "constraint" is a math check over $\mathbb{F}_p$ (e.g. $c - a \cdot b = 0$), not a governance policy. The integer gap is why "ZK receipt" and "bit-exact AE recompute" are different Plan B / Plan A stories.

Constraint system intuition: turn "this forward pass was correct" into a huge set of algebraic checks over a *prime field* $\mathbb{F}_p$ (multiplications, lookups, range checks). The prover knows a *witness* (weights, activations) satisfying every constraint; the verifier checks a short proof that such a witness exists for the public statement — without learning the witness.

Integer constraint, concretely: GPUs run BF16/FP8 with rounding; SNARK circuits run integers mod p. zkLLM-style systems therefore prove a *quantized integer circuit* that approximates the model, not "whatever your CUDA float kernel did today." That is why Plan A cares about bit-exact float recompute, and why VerInf replaces exact equality with an unexplained-info bound ($\hat{U}$ / U-hat). Failures of soundness = false compliance; failures of ZK / confidentiality = IP/data leakage through the proof; failures of assurance = you never sampled the cheat.

1. Wasil et al. – verification method menu

arXiv:2408.16074 · 2024

Who

Wasil, Reed, Miller, Barnett et al. — survey aimed at international AI agreements (access assumptions + evasion sketches).

Maturity

Survey / coordinate system. Not a deployable stack. Useful to prevent “we solved verification” talk after reading one mechanism.

Problem it frames: If states agree “don’t secretly train too big / don’t secretly build too many DCs,” what instruments exist? Two violation types: **unauthorized training** and **unauthorized datacenters**.

Lineage: Entry point for the rest of this deck. Later works sit in its hardware-dependent / access-dependent cells (Shavit, FlexHEG, location proofs, workload monitoring).

A · National Technical Means

- ① remote sensing
- ② whistleblowers
- ③ energy monitoring
- ④ customs · ⑤ finance
- little access needed

B · Access-dependent

- ⑥ DC inspections
- ⑦ fab inspections
- ⑧ developer audits
- need consent / entry
- IAEA-flavored

C · Hardware-dependent

- ⑨ chip location track
- ⑩ on-chip / workload
- pre-agree chip rules
- Shavit / FlexHEG

Swiss-cheese model



each layer has holes; stacked layers make covert breach harder (Fearon / Baker: verifiable agreements are more

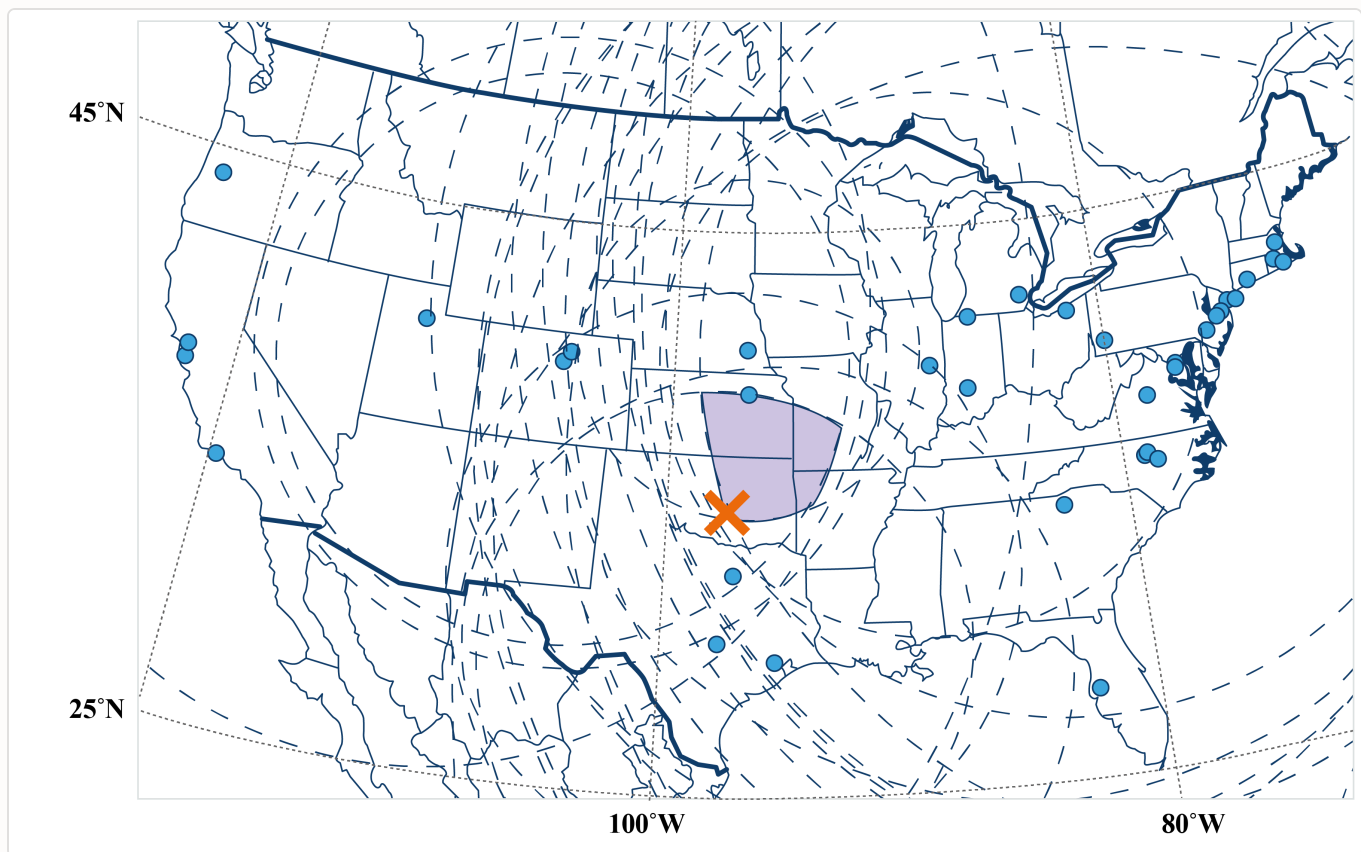
Wasil taxonomy: cooperation cost ↑ left→right; technical bite ↑ left→right. Not a system design — a menu.

What to remember technically

- Methods differ in **access assumptions**: unilateral national technical means vs negotiated entry vs pre-agreed chip behavior.

- Hardware-dependent methods are highest bite and hardest diplomacy.
- Chip location $\neq$ training proof. Location answers “where is the silicon?”; workload methods answer “what did it run?”

One Wasil cell — chip location proofs — looks like this: you are not proving “this GPU trained legally.” You are proving (approximately) “this registered chip is physically near claimed site X,” so a dark training cluster in an undeclared country is harder to hide. Delay-based / multi-anchor checks exploit physics: light in fiber has a max speed, so round-trip time to a trusted probe sets a distance upper bound.



How to read the figure. Blue dots = trusted *anchors* (known-location probes / PoPs). Dashed arcs = “this chip answered me in $\leq T$ ms $\Rightarrow$ it cannot be farther than $\sim c \cdot T/2$ in fiber.” Purple blob = intersection of several such bounds (where the chip is allowed to sit). Orange X = true location (ideally inside the blob). Spoofing “I’m in Virginia” while sitting in another country fails if enough distant anchors force the intersection away from Virginia. This is only a *where* check — still need workload methods (Shavit / joshc / FlexHEG / ...) for *what* ran.

2. Shavit – Catch a Chinchilla (+ Proof-of-Learning)

arXiv:2303.11341 · 2023 · PoL primitive: Jia et al. 2021

Who

Yonadav Shavit — early blueprint for monitoring large training runs via chip-level weight fingerprints + training transcripts + supply-chain coverage.

Maturity

Blueprint. Problem decomposition, not an industry-deployed end-to-end system as of 2026.

One sentence: force frontier training onto registered ML chips that leave sparse *weight fingerprints*; require a transcript that explains those fingerprints (Proof-of-Learning-style checks); audit the supply chain so dark chips can't skip the net.

Lineage: Classical “what Claim A monitoring could look like.” Gaps it leaves → later work: train↔infer ambiguity (Rahman); cryptographic transcripts (Peigné); active hardware caps instead of after-the-fact snapshots (FlexHEG).

① Chip firmware

sparse weight snapshot
secure-boot signed FW

② Transcript + PoL

data / hparams / ckpts
spot-check recompute

③ Supply registry

known ML chips only
spot physical audit

PoL check (Jia et al. intuition)

Claim: W_T came from (W_0, D, A) under claimed optimizer steps
Verifier samples steps $i \rightarrow$ recompute / check consistency with snapshots
Mismatch $\Rightarrow$ lying about process · Match + illegal FLOP $\Rightarrow$ rule break
Refuse audit $\approx$ adverse inference (audit analogy)

Shavit stack: fingerprints on chip + proof the transcript explains them + no dark chips outside the registry.

Threat model that matters

Prover is a **covert adversary**: cheats only when it can look compliant. Design goal = make “look compliant while cheating” nearly impossible under random chip audits.

On chip

Firmware occasionally hashes / records HBM weight shards. Needs signed firmware + later physical spot checks (remote attestation alone is weak against a physical adversary).

In the DC

Transcript = data references, hparams, checkpoints, optimizer trajectory. Verifier samples steps; inconsistency or illegal scale ⇒ fail.

Open issues Shavit already flags

- Train vs infer lookalike occupancy of HBM.
- Legacy chips without secure firmware — small/medium training may escape.
- Vendor–prover collusion on backdoored firmware.

3. Rahman & Tajdari – train vs infer from NVML

arXiv:2606.19262 · TAIGR @ ICML 2026

Who

Robi Rahman (MIRI Technical Governance; ex-Epoch) and Sabiha Tajdari (UVA) — SPAR-originated empirical red-team of GPU workload classifiers.

Maturity

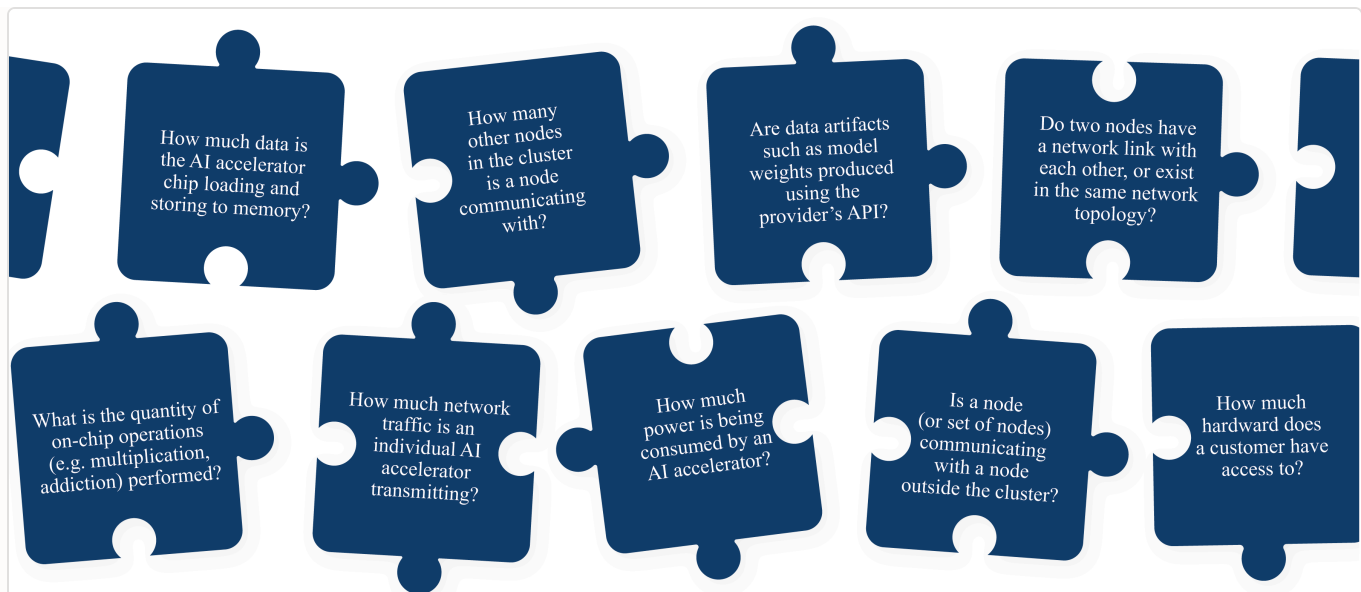
Empirical paper + public tooling lineage. Real numbers under adversarial disguise. Solves only a *thin slice* of Claim A (label train vs infer), not full rule verification.

Question: can you tell training vs inference from GPU telemetry (NVML) without touching weights or data?

Lineage: Attacks the gap Shavit flagged and that FlexHEG/joshc classifiers often assume. Complements interconnect-based detectors (communication volume) — different observation channel.



Thin layer only: “is this training?” not “did this training violate a FLOP license.”
Empirically grounded where Shavit/ZK are still blueprints.



Example metric space for workload monitoring. Rahman asks whether this works under disguise.

Numbers to keep (and not overclaim)

- ~98.2% overall accuracy across their matrix.
- Under the hardest disguises, ~43–87% — robust overall, meaningfully degraded under sophisticated evasion; not “solved.”
- Method: 5-round monitor–evader loop, multi-GPU / multi-arch — the right evaluation bar for any such detector.

4. joshc – TAPs, wipes, and covert RL

LessWrong / Alignment Forum · 2026

Who

joshc (Alignment Forum / LessWrong) — red-team essay on whether *fast, cheap facility mods* can slow training before treaty-grade hardware exists.

Maturity

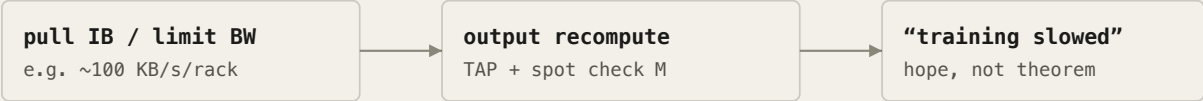
Red-team / requirements doc. Not a product. Sets pass/fail bars for prototypes (TAP + recompute + wipe + accounting).

Question: before global FlexHEG-class silicon exists, can governments slow training by pulling InfiniBand, capping rack egress (~100 KB/s), TAP+recomputing outputs, wiping memory, demanding PoW/PoM?

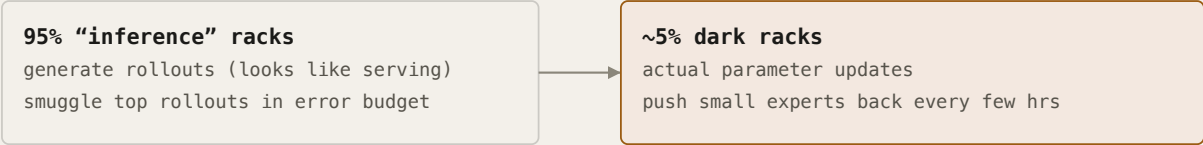
Naive lever: training is communication-hungry; inference is text-skinny. joshc's point: under *today's prototype strength*, that lever may fail against modern RL pipelines (rollouts look like inference; updates can hide in a small dark fraction of compute + leaky recompute slack).

Lineage: Stress-tests the "medium" rung (after you've seen how thin NVML train $\leftrightarrow$ infer is). Motivates tighter recompute (Cankaya bit-exact / DiFR), better taps, and why FlexHEG remains the longer-term enforcement story.

Naive story: training needs fat interconnect; inference needs skinny text



Covert RL under leaky recompute (~1% error budget + ~5% dark compute)



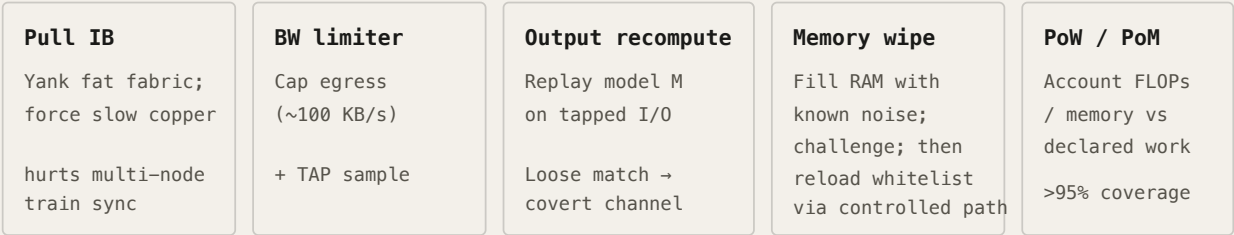
Thresholds to break this (any one, vs current algos)

PoW/PoM covers >95% · memory wipe every few minutes · covert channel <0.01% of traffic

joshc’s killer scenario: compliant-looking RL rollouts + dark updater. Near-term network controls without those thresholds may not buy a real pause.

Facility toolkit (where each tool sits)

joshc near-term toolkit (facility mods before FlexHEG silicon)



Whitelist channel = the only high-bandwidth path allowed to reload weights after a wipe
— gated by Hash(W)∈registered commitments.

Five near-term tools (compressed)

TOOL	IDEA	AUTHOR’S READ
Pull IB	Replace ~400 GB/s fabrics with slow copper	~1y prep plausible
BW limiter + TAP	Cap egress; sample traffic	to ~100 KB/s plausible
Output recompute	Replay declared model M on tapped I/O	~1% nondeterminism budget = covert channel

TOOL	IDEA	AUTHOR'S READ
Memory wipe	Force verifiable fill; reload whitelist model	needs »99.99% coverage — hard
PoW / PoM	Account for FLOPs/memory spent on declared serving	useful if >95% coverage

PROTOTYPE BAR

If a demo doesn't close the RL-smuggling channel under realistic slack, it has not purchased a credible pause — regardless of how good benign accuracy looks.

5. Cankaya – taps, challenges, and precise replay (Plan A)

System overview LW · taps Substack Apr 2026 · bit-exact 2606.00279 · DiFR (statistical slack) 2511.20621

Who

Naci Cankaya (MIRI technical governance) — whole-facility retrofit blueprint: capture I/O → random challenge → evaluate. Bit-exact paper is the Plan A numerics companion; taps note is the capture companion.

Maturity

Overview: **blueprint**. Taps: **feasibility + commodity-tech argument** (NS ready-ish; EW needs pilots). Bit-exact: **empirical + emulator prototype** for inference — not a training verifier; not yet a fielded AE standard.

Shared audit question: was this sampled request run on the whitelist model?

Plan A (near-term default): air-gapped audit environment (AE) holds the whitelist weights and *recomputes* the challenged forward pass in plaintext, then compares to tapped I/O. For that comparison to be a pass/fail (not a ~1% stego channel à la joshc), you need **precise replay** — ideally bit-exact under recorded metadata, or a disciplined statistical alternative (DiFR).

Plan B (later sections): replace AE recompute with ZK so the peer needn’t see weights — zkLLM (§6) or VerInf (§7). Same challenge loop; different evaluate engine.

Lineage: Turns joshc’s TAP/recompute sketches into an arms-control-shaped architecture. FlexHEG is the longer hardware track; Peigné (§9) is the training-ZK cousin.

Properties checklist (before the diagrams)

PROPERTY	PLAIN ENGLISH	PLAN A (AE PLAINTEXT)	PLAN B (ZK)
Completeness	Honest clean prover passes	Needs bit-exact / DiFR so benign noise ≠ reject	Honest π verifies
Soundness	Cheater fails	Recompute mismatch / wrong W vs C	Bad witness can’t forge π (or $\hat{U}$ bound)

PROPERTY	PLAIN ENGLISH	PLAN A (AE PLAINTEXT)	PLAN B (ZK)
Confidentiality	Secrets stay secret	Weak — W and challenged I/O enter AE	Stronger — witness not revealed
Assurance	Field catch probability	Soundness × sample rate × tap coverage × stego slack	Same multipliers; $\pi \neq$ free lunch

Why plaintext AE recompute is not “the endgame”

1 • IP / data

Whitelist W and challenged x,y enter the AE in clear.
Rival states / labs hate this.
→ confidentiality failure

2 • Mutual distrust

Whose GPU is the AE?
Prover won’t trust verifier’s box; verifier won’t trust prover’s box either.

3 • AE is still a TCB

Air-gap $\neq$ zero leak.
Need diodes, cross-checks, physical monitoring –
Cankaya’s hard part.

So why is Plan A still the near-term default?

Because it is fieldable without frontier-scale ZK: if you can stomach AE confidentiality risk + build bit-ex you get real assurance against “swap the model on the wire.” Plan B (zkLLM / VerInf) replaces the AE evaluation; confidentiality is the dealbreaker – it does not remove taps, sampling, or the need for soundness of the state.

Plaintext recompute answers soundness-ish checks by literally redoing the math. ZK answers them with a proof. Same audit question; different confidentiality cost.

Why plaintext recompute is not enough (even when it “works”): it can deliver completeness/soundness/assurance under good engineering, but it fails the low-trust confidentiality goal — rival labs/states will not ship frontier weights into the other party’s AE, and neither trusts the other’s silicon inside the cage. That is the explicit motivation for Plan B. Plan A remains the near-term default because ZK at frontier scale is still expensive / immature — not because confidentiality doesn’t matter.

Same audit question: was this sampled request run on the whitelist model?



Dead-simple memory aid

Plan A: I watch you recompute.

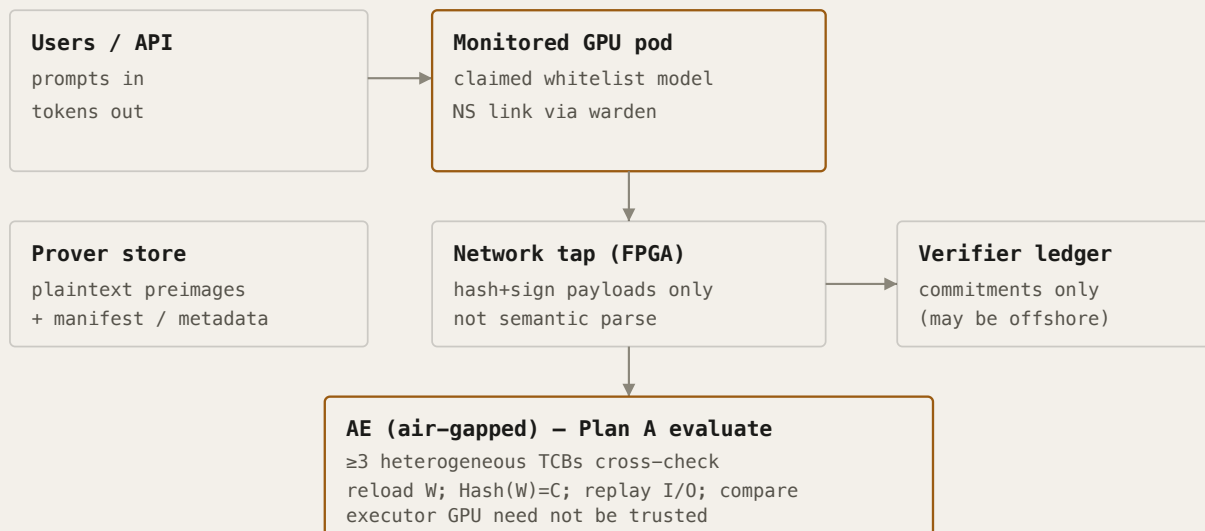
zkLLM: you prove the circuit executed (receipt).

VerInf: you prove the committed model explains the tokens; leftover $\leq \hat{U}$ (score).

Near-term default in Cankaya: Plan A. zkLLM = early Plan-B instance. VerInf = frontier-scale Plan-B core.

Inference side of Claim A / arms-control-like monitoring. Complements Peigné's training ZK; sits under Cankaya's tap → challenge → evaluate loop.

Cankaya Plan A sketch – taps + air-gapped audit environment (AE)



Adapted from Cankaya's system overview: online evidence capture is continuous; evaluation is random challenges. Plan B swaps the AE plaintext recompute for a ZK proof.

Cankaya's four doors on a challenged forward pass

1 · Weights

Hash(W) = C
whitelist model?
Hard check

2 · I/O evidence

preimage matches
tap commitment
Hard check

3 · Recompute

run W on input;
output matches?
Hard (bit-exact ideal)

4 · Semantics

blacklist use?
(CBRN, etc.)
Soft / inspectors

Doors 1–3 can all pass while door 4 fails: whitelist model doing banned things. Soft classifiers are not a substitute for 1–3.

Network taps (where commitments come from)

Where to put network taps (Cankaya Apr 2026 feasibility note)

North–South (frontend)

DC ↔ outside world (API, SSH)
cheap · active warden OK
cost ≪ 0.01% of facility TCO
do this first

East–West (backend)

GPU↔GPU · storage · all-reduce
optical split + random sample
~0.2–1.5% TCO (level-dep.)
copper scale-up (NVL*) stays blind

Tap outputs commitments (hash + timestamp + sign), not raw secrets to the peer state.

Prover can passively re-observe tap egress → near-term no mutually trusted silicon.

Storage fabric next (~0.3–0.5%): which weights loaded, datasets read, checkpoints written.

Biggest unknown: residual covert bps if only NS is tapped (Fermi ~10² bps – needs empirics).

Evidence capture layer under Cankaya's blueprint. Source: Cankaya, secure network taps (Apr 2026).

- **NS first:** frontend traffic is ~Gb/s-class in the note's Fermi — commodity FPGA / mid-range NIC; active wardens fit.
- **EW:** full mirror is infeasible; wide optical split + random sampling; need tensor metadata or bytes are meaningless.
- **Copper NVL*:** passive optical taps miss intra-rack scale-up — security falls back onto NS + storage.

Bit-exact inference verification (precise replay)

This is the load-bearing Plan A engineering claim: unmodified serving stacks are often *deterministic enough* once you freeze the right metadata — so “looks like noise” is usually non-invariance (reduction trees), not irreducible chaos.

Plan A numerics: close joshc’s “~1% looks like error” channel

Non-invariance (common)

Same math, different FP reduction trees
(batch / kernel / SKU) → different bits
Deterministic if metadata fixed

True nondeterminism (rare)

atomicAdd races (some INT dequant)
Warp-scheduler order changes
Avoid / ban those kernels for audit

Record: SKU · weights · parallel topology · software stack · batch size / step

Then recompute on same SKU, or CPU-emulate tensor-core + SFU rounding (cross-gen).

No need for slow “determinism flags” on the live path – audit via metadata + replay.

Contrast: DiFR = statistical slack; VerInf = don’t require float bit-exact (Ü absorbs).

arXiv:2606.00279 — bit-exact inference verification (Cankaya).

- Unmodified vLLM/HF are usually **deterministic** without special determinism flags; “noise” is mostly **non-invariance** (reduction trees), not atomics.
- CPU emulator can match A100/L40/H100 intermediates when SKU + kernel path are modeled.
- **Metadata checklist:** SKU · weights · parallel topology · software stack · per-forward batch size.
- **DiFR:** if you cannot field bit-exact, keep statistical slack — but then budget the covert channel explicitly (joshc’s warning).

HOW THE PIECES LOCK

Tap → I/O commitments · Bit-exact (+ metadata) → close stego slack · AE recompute (Plan A) evaluates samples. §§6–7 swap in ZK at the evaluate step only.

6. zkLLM – prove the integer circuit ran

arXiv:2404.16109 · Sun, Li, Zhang · ACM CCS 2024 · code

Who

Haochen Sun, Jason Li, Hongyang Zhang — first widely cited exact-circuit ZK system for LLM inference at non-toy scale (~13B).

Maturity

Published system + limited scale. Spot-check engine, not “every API request.” Integer/quantized circuit $\neq$ arbitrary float serving bit-exact.

Statement: there exist weights W matching public commitment C such that the public architecture, run as an *integer circuit* on public x , yields public y — without revealing W or activations.

Lineage: One Plan B evaluate backend under Cankaya’s challenge loop. Precursor / contrast to VerInF (§7): exact receipt vs unexplained-info score. Complements Plan A when the AE cannot see weights.

⑤ full LLM: layer stack $\rightarrow$ public x, y + Commit(W)= C

④ zkAttn: Softmax shift + digit exp tables + norm tolerance

③ tlookup: non-arith ops $\rightarrow$ set membership via rational identity

② sumcheck: matmul / big sums via random challenges

① Hyrax poly commit: hide W , bind C , ProveEval at random points

zkLLM composition: verifier never re-runs the model — only checks random algebraic relations against commitments.

Numbers (paper anchors)

ITEM	ROUGH PUBLIC FIGURE
Scale	~13B (OPT / LLaMA-2 class)
Prove time	~1–15 min (CUDA, setting-dependent)
Proof size	< ~200 KB
Verify	~1–3 s
Commit scheme	Hyrax (no trusted setup)

Guts (enough to not handwave)

- **Hyrax:** polynomial commitment binds W into C ; open evaluations at random points.
- **Sumcheck:** big matmuls / sums become interactive algebraic checks.
- **lookup:** non-arithmetic ops $\rightarrow$ prove secret tensor entries lie in a public table via a rational identity at a random point.
- **zkAttn:** Softmax shift shrinks exp domain; don't prove exact log-sum-exp — only a **tolerance band** on row sums (fine for execution receipts; dangerous if bolted onto a manipulable unexplained-info score).

What it does *not* prove

- Bit-exact match to H100/B200 float serving (that's Plan A / bit-exact paper).
- Anything about training — only a forward pass receipt.
- Frontier 100B+ scale in the published system (see VerInf for the larger demo with a different statement).

CHEAT INTUITION

Swap whitelist model $\rightarrow$ fails vs C . Abuse Softmax tolerance $\rightarrow$ can still Accept under loose bands. Float nondeterminism is mostly irrelevant because you never claimed float equality.

7. VerInf – bound leftover bits, skip float bit-exact

Petrie 2026 draft · github.com/JamesPetrie/VerInf · FLI / FlexHEG author

Who

James Petrie — same hardware-governance thread as FlexHEG. VerInf is the “replace the recompute cluster with a ZK evaluate engine” instance at frontier demo scale.

Maturity

Working draft + large demo, not production audit tooling. Interactive, expensive proofs meant for rare high-stakes spot checks when both parties have serious compute.

Statement: a committed integer model explains a (possibly hidden) token stream well enough that leftover unexplained information is $\leq$ a public bound $\hat{U}$ bits/token — without revealing weights, activations, or plaintext I/O.

Lineage: Other Plan B evaluate backend under Cankaya. Designed precisely because Plan A bit-exact and zkLLM-style exact equality both get brittle or tiny-scale at frontier serving.

VerInf statement (not bit-exact equality)

Committed integer model M

predicts hidden token stream
well enough that leftover
info $\leq$ public $\hat{U}$ bits/token
ZK: no weights / acts / x, y

Demo anchor (Petrie 2026)

Llama-4-Maverick $\sim 400B$ MoE
 1000 tokens $\cdot \hat{U} \approx 0.88$ b/tok
 $\pi \approx 94$ GB $\cdot$ prove $\sim 14h$ $\cdot$ verify $\sim 18h$
spot-check scare, not every request

vs zkLLM

zkLLM: prove exact integer circuit output y . Softmax via tables + tolerance. Scale $\sim 13B$, tiny π .
VerInf: prove predictive explanation bound. Deliberately skips float bit-exact. Scale $\sim 400B$, huge π .
Both are Cankaya Plan B evaluate engines — after taps/challenge exist. Plan A stays the near-term default.

VerInf — unexplained-info ZK at frontier demo scale.

Why change the statement

- Float nondeterminism + MoE/serving stacks make “y must equal AE recompute” expensive to field at 400B.
- Exact-circuit ZK at 13B already hurts; naively scaling the zkLLM statement is the wrong lever.
- $\hat{U}$ is an auditable score: too large $\Rightarrow$ looks like wrong model / stego / garbage; threshold is a policy knob.

Demo anchors (draft)

ITEM	ROUGH FIGURE
Model	Llama-4-Maverick ~400B MoE (commit all experts/layer)
Sequence	1000 tokens (500+500), fully hidden in the demo
$\hat{U}$	≈ 0.88 bits/token (~95% of vocab info explained)
Prove / verify	~14 h prove · ~94 GB π · ~18 h verify (single high-end box class)
Witness stream	~7 TB streamed · ~84 GB peak RAM

vs zkLLM (cheat table)

CHEAT	PLAN A (§5)	ZKLLM (§6)	VERINF (§7)
Swap whitelist model	recompute mismatch	fails vs C	$\hat{U}$ blows up
Float nondeterminism	pain; bit-exact / metadata	proves circuit, not float replica	slack $\rightarrow \hat{U}$
Softmax slack abuse	if loose match, channel	still Accept possible	forbidden design goal; unique shift

COST INTUITION

Spot checks, not every request. Plan A $\approx$ recompute GPU-time. zkLLM-13B ~minutes + tiny π . VerInf-400B = hours + tens of GB — rare audits when secrecy of weights beats AE plaintext.

8. FlexHEG trilogy – especially Part II

Tech 2506.03409 · Overview 2506.15093 · Intl 2506.15100 · flexheg.com · 2025

Who

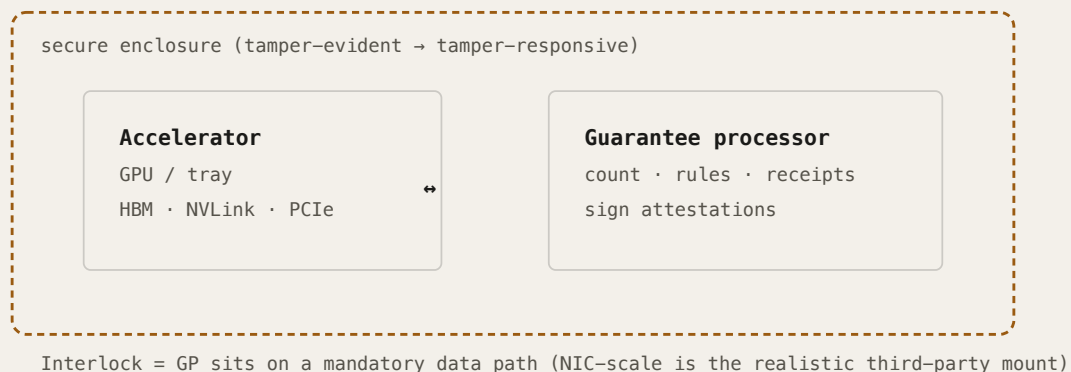
James Petrie, Onni Aarne, and collaborators
(FLI / ARIA-linked FlexHEG program).
Hardware-enabled governance design
for accelerators.

Maturity

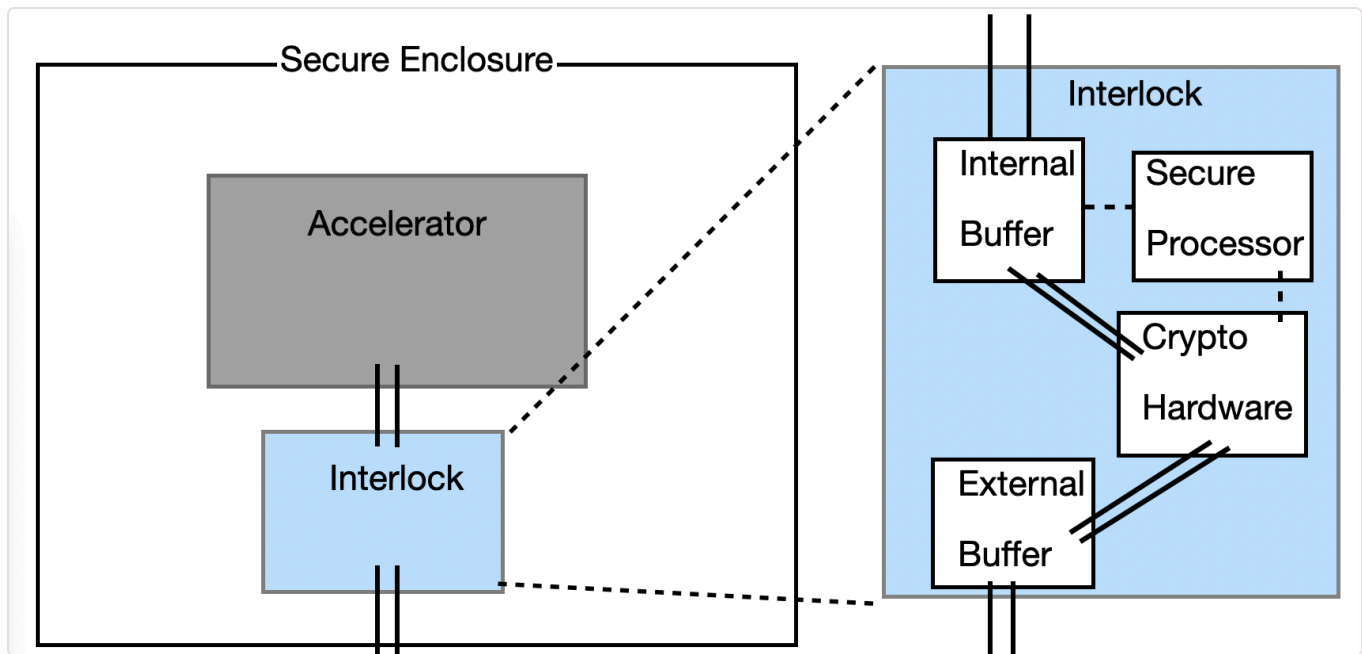
Blueprint + early demos (enclosure / metering ideas). **Not** mass-deployed product silicon.
Grant-funded engineering (incl. partners building tap/wipe primitives).

FlexHEG = Flexible Hardware-Enabled Guarantees: a **guarantee processor** that locally enforces / attests rules, plus a **secure enclosure** that makes bypass expensive. Rules should be updatable under multi-party authorization.

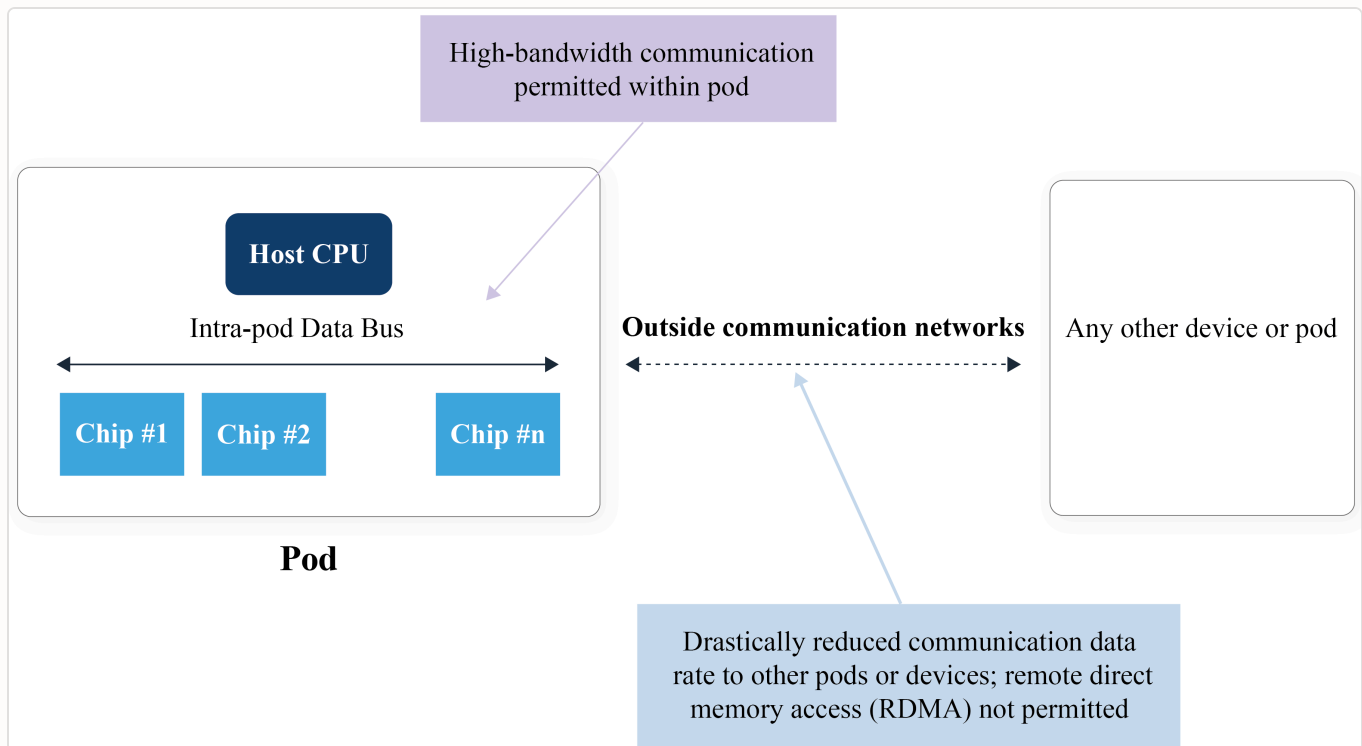
Lineage: After the near-term retrofit path (§§5–7): moves Claim A from Shavit-style after-the-fact snapshots toward *active* guarantees (FLOP caps, licenses, interlocks). Near-term retrofit path often emphasized: NIC / scale-out interlock before in-die changes.
Complements Cankaya’s facility-level taps (different layer).



FlexHEG dual core: local notarization + physical hardness. Privacy: prove compliance without shipping weights abroad.



Interlock intuition: put the guarantee processor on a path data must cross.

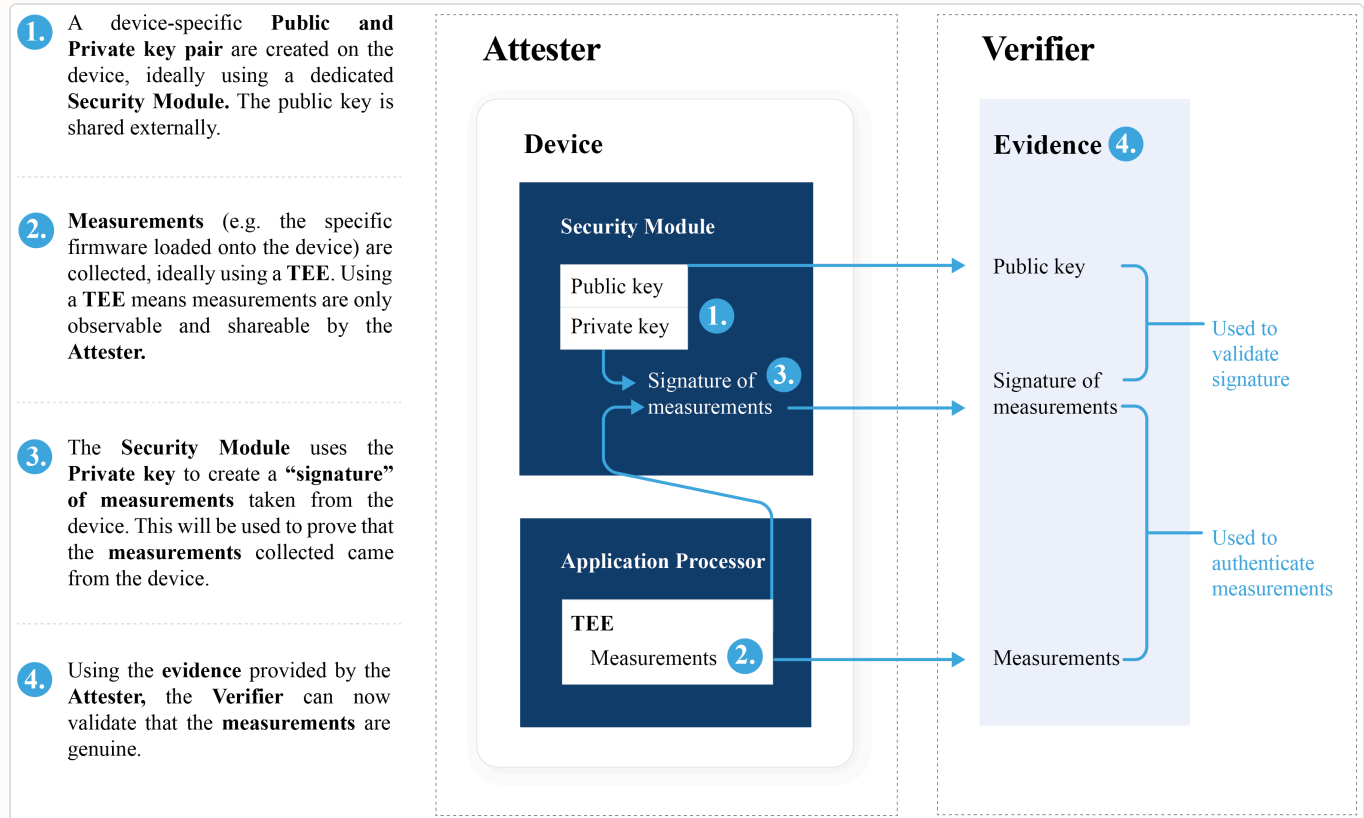


Fixed-set pod idea: fast links inside a declared set; starved RDMA outside — a governance-shaped topology.

Integration ladder (speed vs hardness)

MOUNT	DEPLOY SPEED	SECURITY	WHO CAN SHIP IT
In-die / PCB redesign	slow (multi-year silicon + fleet churn)	strongest	incumbent vendors
Retrofit modules	medium	medium (detach risk)	third parties possible

MOUNT	DEPLOY SPEED	SECURITY	WHO CAN SHIP IT
Firmware-only	faster	weaker vs physical	needs signed secure boot
Pure software	fastest	weakest	anyone — not a treaty root



Attestation mental model — local measurement → signed statement → remote verify.

9. Peigné-Lefebvre et al. – ZK verification of frontier *training*

arXiv:2606.05433 · 2026 · GPAI Policy Lab / FAR.AI · Sorbonne/LIP6

Who

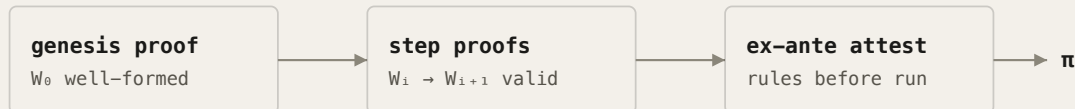
Pierre Peigné-Lefebvre, Ky Nguyen, and coauthors — argument that ZK verification of frontier *training* is possible in principle via a BF16/FP32-aware zkVM.

Maturity

Horizon. Authors are explicit: no working end-to-end prototype; rough ~36 months to a deployable PoC vs 6–10 years for custom verification silicon; long open-problem list.

Idea: make Shavit’s transcript story cryptographically binding — genesis / step / optional ex-ante proofs without revealing the proprietary run.

Lineage: Different problem than §§5–7 (those audit *inference* whitelist forward passes). This is the crypto endgame of classical *training* monitoring. Does not replace near-term FlexHEG/Cankaya work; clarifies what “strong” would mean if telemetry stays porous.



vs Shavit

Shavit: heuristic / sampling recompute of a claimed transcript

ZK: cryptographic proof over BF16/FP32 training steps in a zkVM – stronger claim, no E2E prototype yet

Authors: ~36 months to deployable PoC; custom verify silicon 6–10 years; 13 open problems listed

Crypto endgame of the Shavit idea. Treat as research horizon, not near-term pause tooling.

10. Putting the stack in one breath

What an outsider should be able to repeat after this deck.

Claim A = verify no covert frontier training ($\neq$ TEE model-identity or confidential-cloud claims B/C). Wasil is the method menu. Shavit is the classical chip+transcript blueprint. Rahman measures how far NVML train $\rightarrow$ infer detectors survive disguise (closes Shavit's lookalike gap). joshc red-teams cheap facility controls against covert RL.

Cankaya is the near-term low-trust retrofit: NS-first taps feed commitments; Plan A = AE precise replay (bit-exact metadata; DiFR if needed). Plan B swaps the evaluate step for ZK – zkLLM (~13B exact integer circuit) or VerInf (~400B unexplained-info bound $\hat{U}$). FlexHEG is the parallel hardware-enforcement track (active caps / interlocks), not a substitute for taps+evaluate. Peigné sketches ZK *training* as a longer crypto horizon. Almost none of this is mass-deplo empirics, and prototypes.

Open problems / next-step research (what to do after this deck)

These are not “unsolved math curiosities” — they are the load-bearing gaps between today's blueprints and a credible pause. Several track cruxes raised against Cankaya's retrofit (precise replay, sampling economics, perimeter, rare covert adapters, Plan A $\rightarrow$ B transition, adjudication).

#	PROBLEM	WHY IT BLOCKS	CONCRETE NEXT STEP
1	Precise replay vs softer accept	Plan A soundness/completeness hinge on bit-exact (or DiFR slack). Unknown: residual mismatch rate across SKU/driver/batch, and FP cost of chasing zero mismatch vs budgeting a channel.	Medium-scale empirics (e.g. ~70B) across heterogeneous GPUs/drivers: exact-match rate vs DiFR-style accept; metadata necessity set; cost/latency of each policy.
2	Sampling economics	Assurance = soundness $\times$ sample rate. Appendix-style Fermi needs a real BoM: can Plan A / zkLLM / VerInf hit ~1% overhead at a stated catch probability?	Publish cost curves: challenge rate $\rightarrow$ catch prob $\rightarrow$ \$/request for AE recompute vs zkLLM-13B vs VerInf-400B-class proofs.
3	Perimeter / NS-only residual	NS-first taps leave EW copper and off-perimeter compute undertapped. Bar: residual outside the fence can't do	NS-only red-team budget (joshc-style local RL / DiLoCo): how many covert FLOPs survive;

#	PROBLEM	WHY IT BLOCKS	CONCRETE NEXT STEP
		consequential covert work at high confidence.	optional interconnect detectors (Fowler-style) as complement.
4	Rare-trigger / thin adapters	Forward-pass recompute can match a whitelist while a rarely activated LoRA / tool path does harm — identity ≠ all governance goals.	Red-team: memory probes, NVML anomaly, wipe/accounting, inspector agents under disclosure bounds; measure miss rate for sparse triggers.
5	Train↔infer under disguise	Rahman: hard disguises degrade NVML classifiers. joshc: RL rollouts look like serving. Soft labels alone won't carry Claim A.	Combined channel study: NVML + interconnect bytes + tap commitments under adaptive evade; report ROC under the hardest loop, not only benign accuracy.
6	Plan A → Plan B transition	Plaintext AE burns confidentiality; ZK restores it but is immature/expensive. Stakeholders need a migration path, not a cliff.	Timeline + dual-run design: which challenge classes stay on AE, which move to zkLLM/VerInf as prove-time falls; joint ops with both evaluate engines.
7	Physical security of AE / taps	Low-trust Plan A assumes rival TCBs + continuous physical monitoring. If forgeable silently, assurance collapses.	Red-team / OSINT: diodes, FPGA wardens, SCIFs at scale, hosting foreign TCBs on sensitive weights — map failure modes to FlexHEG hardware track.
8	Adjudication & consequences	Technical Accept/Reject ≠ treaty breach. Without an escalation ladder + false-positive budget, either everything is excused or everything detonates.	Design CBRN-like ladder: anomaly → joint review → deeper disclosure → supervised inspection → predetermined response; name who adjudicates.
9	Hardware enforcement demos	FlexHEG remains the long enforcement story; near-term needs NIC/enclosure interlocks with honest overheads and multi-party rule updates.	Ship a NIC-level license/FLOP-cap prototype; measure overhead; stress single-vendor root-of-trust objections.

HOW TO READ THIS LIST

1–5 are mostly empirical / systems (can move in months). **6–7** mix engineering with deployment politics. **8** is institutional. **9** / §8 FlexHEG is the parallel hardware track. A credible pause needs progress on several in parallel — closing only crypto or only taps is not enough.

PRIMARY SOURCES (PUBLIC)

Wasil 2408.16074 · Shavit 2303.11341 · joshc LW · FlexHEG flexheg.com · Rahman 2606.19262 · Cankaya overview LW · taps Substack · bit-exact 2606.00279 · zkLLM 2404.16109 · VerInf GitHub · DiFR 2511.20621 · Peigné 2606.05433

